212-Корепин-отчёт-Задание 6

Что сделано

Пять программ на C++, каждая в своей папке, каждая из трёх файлов: main.cpp, functions.h, functions.cpp.

- **01_mst_classic** — минимальное покрывающее дерево классическим перебором (Прим), матрица расстояний D, двоичная матрица B, волна при разных порогах.
- **02_mst_grid** — то же дерево квадратно-гнездовым методом (Краскал по своему и 8 соседним квадратам).
- **03_kmeans** — k-means в двух реализациях: классической и квадратно-гнездовой.
- **04_dbscan** — фильтр DBSCAN: D → B → фильтр по K → B* → волна; муравьиные тропы.
- **05_forel** — ФОРЭЛ: два уровня и волна по центрам тяжести.

Во всех программах параметры читаются из файла config.txt, а не задаются в коде. В каждой папке есть тесты: команда test внутри программы и скрипт run_tests.sh, который собирает программу, прогоняет команды из tests.txt и рисует картинки gnuplot.

Как устроены программы

Интерфейс (main.cpp) читает config.txt и файл конусов, создаёт Control и передаёт ему команды пользователя. Control — диспетчер: проверяет команду и раздаёт поручения исполнителям, сам ничего не считает. Каждый исполнитель отвечает за одно дело. В functions.h у каждого класса комментарий: за что отвечает и какими классами пользуется.

Класс	За что отвечает	Пользуется
Config	читает config.txt, отдаёт значения по имени	—
Field	создаёт и хранит точки гауссов под конусами (в DBSCAN — и тропы)	Cone, GPoint
DistMatrix (D)	расстояния между всеми парами точек; считается один раз, её берут и B, и дерево	Field
ByteMatrix, BitMatrix (B)	двоичная матрица графа в байтах и в битах; методы одинаковые	—
GraphBuilder	строит B по D и порогу	DistMatrix, BMatrix
Wave	волна: связные компоненты B; хранит пометки a(i) и номера кластеров	BMatrix, Field
SpanningTree	минимальное покрывающее дерево	DistMatrix (и Field для сетки)
Indicator	номер кластера каждой точки в k-means	—
PointSearch: ClassicSearch, GridSearch	поиск ближайшего центра, общий интерфейс	Field, Indicator
KMeans	алгоритм k-means	Field, PointSearch, Indicator
DbscanFilter	базовые, периферийные точки и шум; строит B*	BMatrix
Forel	один уровень формального элемента	Field
Plotter	пишет данные и скрипты gnuplot, ничего не считает	Field и результаты
Tester	проверки классов, печатает OK или ОШИБКА	проверяемые классы
Control	диспетчер команд	все исполнители

Матрица D полная: в ней есть расстояние для каждой пары точек. Она симметричная, поэтому хранится только половина над диагональю. На 20 000 точках это 763 МБ. Матрица B сначала была в байтах (ByteMatrix), потом её заменили на биты (BitMatrix) одной строкой `typedef BitMatrix BMatrix;`, остальной код не менялся. Тест проверяет, что

обе матрицы дают один и тот же граф. На 1000 точках B в байтах занимает 0,95 МБ, в битах — 0,12 МБ.

1. Дерево: перебор и квадраты

Пять облаков, по 200, 1000, 2000 и 4000 точек под каждым. Сетка 80 × 80, квадрат 0,25. Время построения дерева без учёта D, которая считается один раз при генерации точек. Кластеры — части дерева после удаления рёбер не короче порога 0,4, крупнее 10 точек.

Без оптимизации (как собирает build_all.sh в аудитории):

Точек	Перебор, мс	По квадратам, мс	Во сколько раз быстрее	Кластеров: перебор / квадраты
1000	31,0	1,2	26,1	5 / 5
5000	718,9	27,1	26,6	5 / 5
10000	3087,0	119,1	25,9	5 / 5
20000	13759,8	528,4	26,0	5 / 5

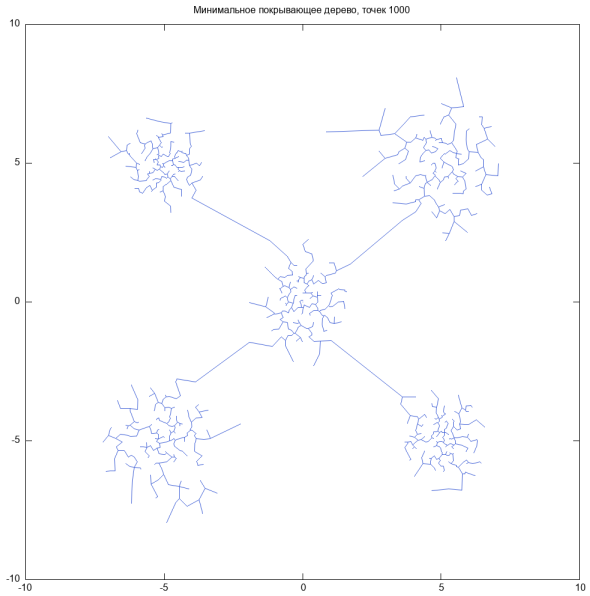
С оптимизацией -O2:

Точек	Перебор, мс	По квадратам, мс	Во сколько раз быстрее	Кластеров: перебор / квадраты
1000	4,1	0,6	6,7	5 / 5
5000	55,7	12,4	4,5	5 / 5
10000	190,0	45,9	4,1	5 / 5
20000	799,1	188,4	4,2	5 / 5

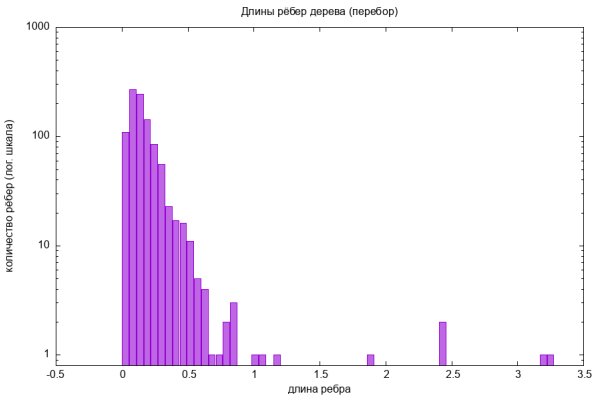
Квадратно-гнездовой метод быстрее в 4–7 раз с оптимизацией и в 26 раз без неё. Кластеры у обоих способов одинаковые. Ускорение зависит от сетки: чем мельче квадраты, тем меньше пар-кандидатов, но дерево распадается на куски. Ниже то же сравнение для разных сеток (-O2):

Сетка	Квадрат	1000 точек, мс	20 000 точек, мс	Кластеров на 1000 / 20 000
80 × 80	0,25	0,8	182,5	5 / 5
100 × 100	0,2	0,3	115,9	6 / 5
160 × 160	0,125	0,2	49,5	10 / 5

При сетке 160 на 1000 точках облака рвутся на куски, поэтому в config.txt оставлена сетка 80.

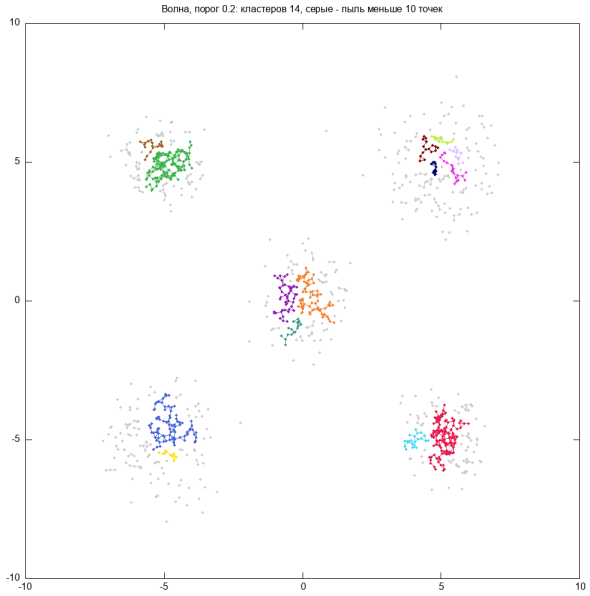


Дерево на 1000 точках

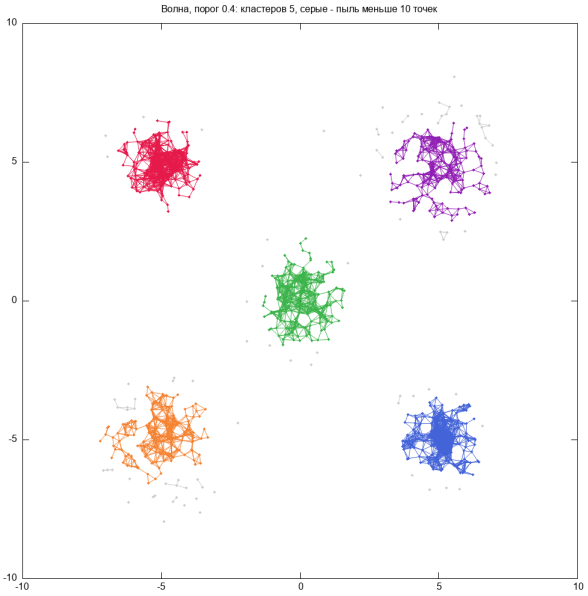


Гистограмма длин рёбер: справа за провалом — длинные рёбра между облаками

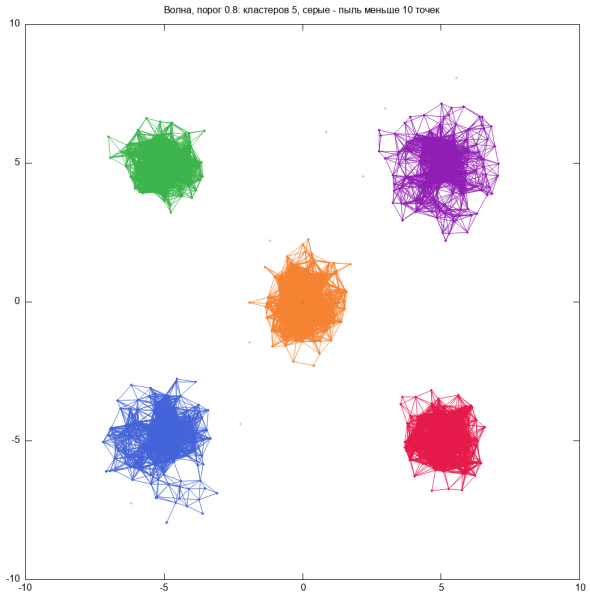
Волна при разных порогах даёт список списков кластеров. При пороге 5 все кластеры сливаются в одну компоненту:



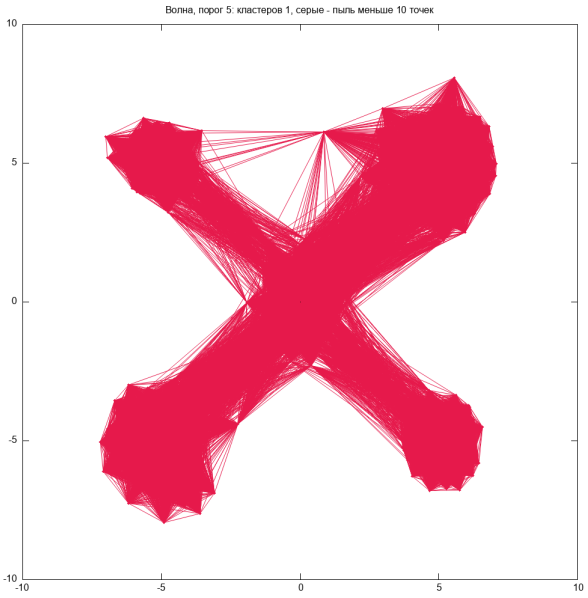
Порог 0,2: 14 кластеров



Порог 0,4: 5 кластеров



Порог 0,8: 5 кластеров



Порог 5: одна компонента

```
> Конусов: 5, точек: 1000
Матрица D: 1.90544 МБ, посчитана за 6.711 мс


> Проверки на 1000 точках:
  OK      матрица D симметрична и считает расстояния верно
  OK      B в байтах и B в битах совпадают
          память B: байты 0.953674 МБ, биты 0.119209 МБ
  OK      волна: каждая точка ровно в одном кластере
  OK      волна: рёбра не выходят за кластер
  OK      дерево: n - 1 ребро и все точки связаны
  OK      дерево, разрезанное по порогу, = волна по тому же порогу


> Дерево перебором (Прим): время 29.547 мс, рёбер 999, сумма длин 178.968
Кластеры: убираем рёбра не короче 0.4 -> частей 61, из них крупных 5, пыли 56
Рисунки: gnuplot plot_tree.gp plot_hist.gp


> Порог   рёбер в B   компонент   кластеров   пыль
0.2      1511      270      14      256
0.4      6144      61      5      56
0.8      22041      13      5      8
5      104200      1      1      0
Список списков: в нём 4 (по одному на порог).
Рисунки: gnuplot plot_wave_*.gp


> Работа завершена.
```

```
> Конусов: 5, точек: 1000
Матрица D: 1.90544 МБ, посчитана за 7.252 мс


> Проверки на 1000 точках:
  OK     матрица D симметрична и считает расстояния верно
  OK     B в байтах и B в битах совпадают
         память B: байты 0.953674 МБ, биты 0.119209 МБ
  OK     волна: каждая точка ровно в одном кластере
  OK     волна: рёбра не выходят за кластер
  OK     лес по квадратам без циклов, длины из D
  (порог больше стороны квадрата 0.25 - сравнение с волной пропущено)


> Дерево по квадратам сетки 80x80 (Краскал): время 1.607 мс, рёбер 949, кус-
ков 51, сумма длин 140.316
Кластеры: убираем рёбра не короче 0.4 -> частей 69, из них крупных 5, пыли 64
Рисунки: gnuplot plot_tree.gp plot_hist.gp


> Порог   рёбер в B   компонент   кластеров   пыль
0.2     1511     270     14     256
0.4     6144     61     5     56
0.8     22041     13     5     8
5     104200     1     1     0
Список списков: в нём 4 (по одному на порог).
Рисунки: gnuplot plot_wave_*.gp

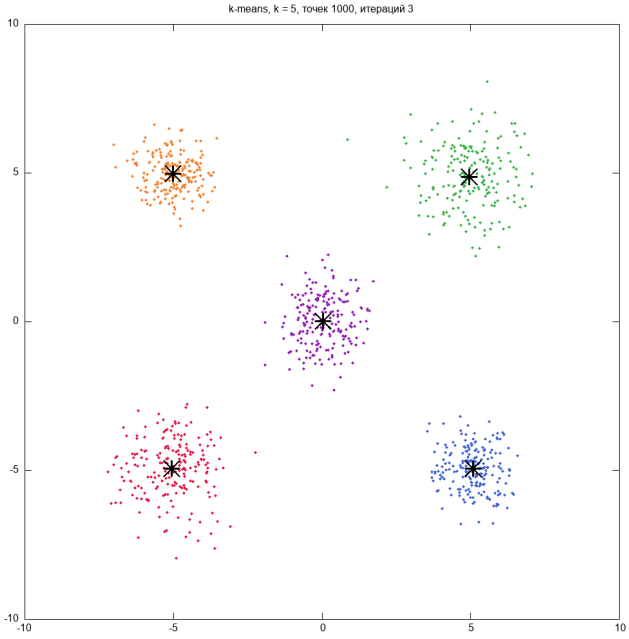

> Работа завершена.
```

2. k-means

Расстояние в обеих реализациях обычное евклидово. Квадратно-гнездовой поиск размечает поле сеткой 20 × 20. Если у всех четырёх углов квадрата ближайший центр один и тот же, у всех точек внутри квадрата он тоже тот: области ближайшего центра выпуклые. Тогда расстояния для этих точек не считаются. По одной считаются только точки в пограничных квадратах. Начальные центры: точка 0, потом каждый раз самая далёкая от уже выбранных.

Точек	Итераций	Классический, мс	Квадратный, мс	Расстояний: классический	Расстояний: квадратный
1000	3	0,026	0,056	15000	6870
5000	3	0,263	0,112	75000	7665
10000	3	0,394	0,294	150000	10215
20000	4	1,017	0,491	400000	16065

Обе реализации раздают точкам одни и те же кластеры, это проверяет тест. На 20 000 точках квадратный поиск считает расстояний в 25 раз меньше и работает вдвое быстрее: остальное время уходит на пересчёт центров.



k-means, k = 5: крестики — центры

```
> Конусов: 5, точек: 1000


> Проверки на 1000 точках:
  OK     классическая и квадратно-гнездовая реализации совпадают
  OK     каждая точка - у ближайшего центра
  OK     каждый центр - среднее своих точек
  OK     квадратно-гнездовой поиск считает меньше расстояний

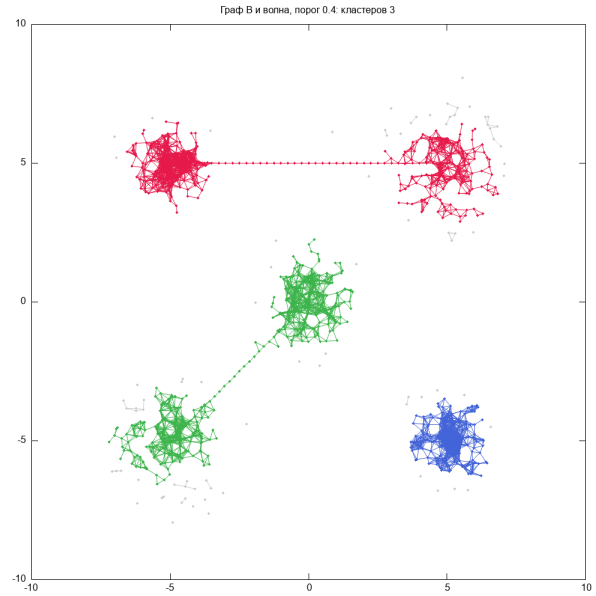

> k-means, поиск классический: k = 5, итераций 3, время 0.405 мс, расстояний
15000
k-means, поиск квадратно-гнездовой: k = 5, итераций 3, время 0.394 мс, рас-
стояний 6870
  центр 1: (-5.06196, -4.93705)
  центр 2: (4.95915, 4.87678)
  центр 3: (5.08019, -4.9368)
  центр 4: (-5.02232, 4.9726)
  центр 5: (0.0354666, 0.031453)
Рисунок: gnuplot plot_kmeans.gp


> Работа завершена.
```

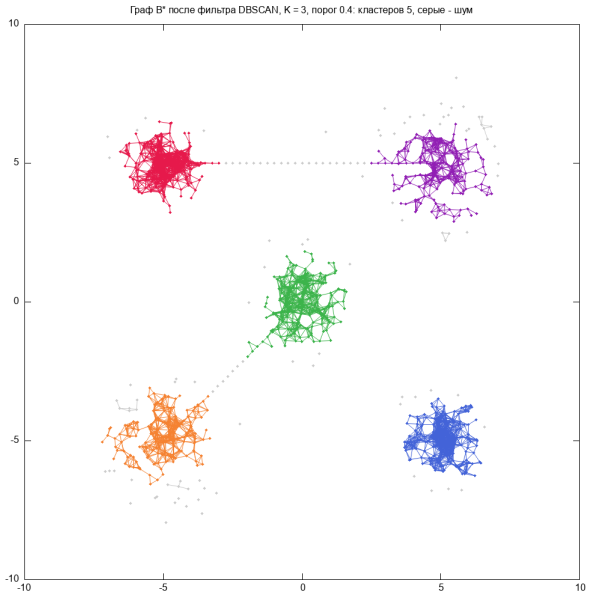
3. DBSCAN

Между облаками добавлены две муравьиные тропы с шагом 0,25, меньше порога 0,4. Обычная волна склеивает облака по тропам и находит 3

кластера. Фильтр DBSCAN при K = 3, 4, 5, 6 разрезает тропы, и кластеров снова 5.



Граф B: 3 кластера



Граф B*, K = 3: 5 кластеров

Точек	**D, мс**	**B, мс**	**Фильтр, мс**	**Волна, мс**	**Кластеров по B**	**Кластеров по B***
1070	0,4	0,6	0,5	0,4	3	5
5070	15,4	13,8	6,1	4,3	3	5
10070	36,8	40,3	17,7	13,3	3	5
20070	204,0	150,1	73,6	47,0	3	5

```
> Конусов: 5, точек вместе с тропами: 1070
Матрица D: 2.18168 МБ, посчитана за 6.298 мс

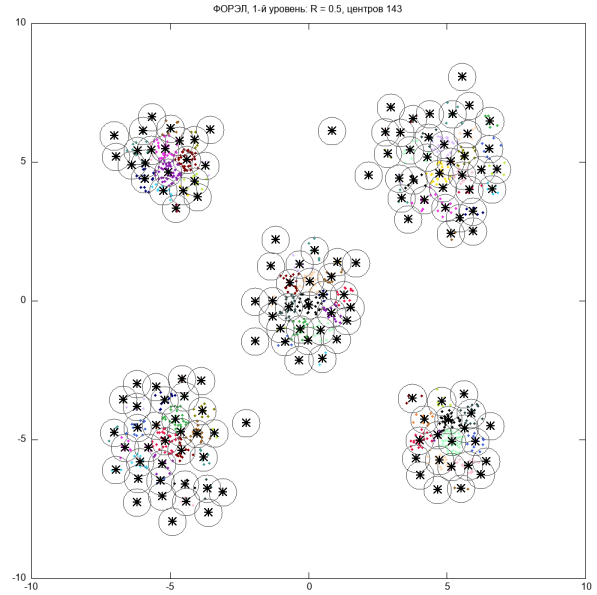

> Проверки на 1070 точках:
  OK     матрица D симметрична и считает расстояния верно
  OK     B в байтах и B в битах совпадают
         память B: байты 1.09186 МБ, биты 0.136738 МБ
  OK     волна: каждая точка ровно в одном кластере
  OK     волна: рёбра не выходят за кластер
  OK     базовые, периферийные и шум - по определению
  OK     в B* только рёбра от базовых точек
  OK     в каждом кластере B* есть базовая точка
```

```
> Обычная волна по B, порог 0.4: рёбер 6618, кластеров 3, пыли 53
Рисунок: gnuplot plot_graph_B.gp

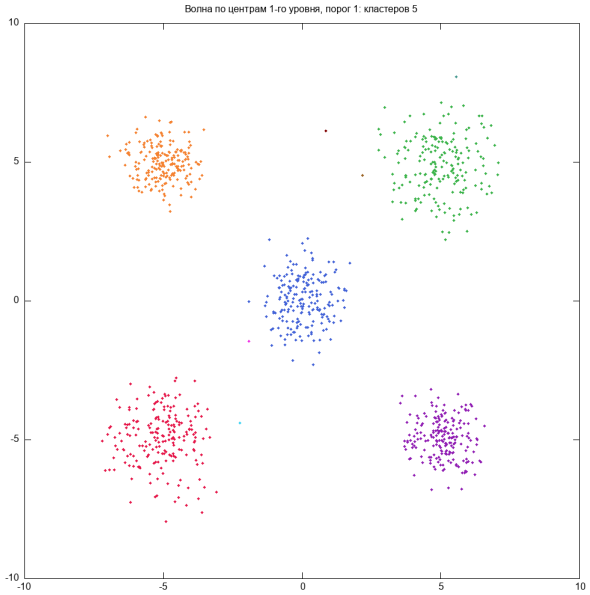

> Порог 0.4, обычная волна по B: кластеров 3
K   базовых   периферийных   шум   рёбер в B*   кластеров
3   918   55   97   6560   5
4   869   70   131   6507   5
5   837   82   151   6464   5
6   787   110   173   6403   5
Рисунки: gnuplot plot_graph_Bstar_K*.gp


> Работа завершена.
```

4. ФОРЭЛ



1-й уровень, R = 0,5



Волна по центрам 1-го уровня

Точек	R	Центров 1-го уровня	Центров 2-го уровня	Время, мс
1000	0,5	143	40	0,3
5000	0,5	204	53	5,2
10000	0,5	248	61	14,8
20000	0,5	277	64	31,2

На 10 000 точках: при R = 0,5 волна по центрам находит 5 кластеров, при R = 1 — 3, при R = 1,5 — 4. С большим радиусом редкие центры на краях облаков соединяют облака.

```
> Конусов: 5, точек: 1000


> Проверки на 1000 точках:
  OK     каждая точка в одной группе
  OK     точки группы внутри круга радиуса R
  OK     сумма весов центров = числу точек
  OK     второй уровень: центров меньше, вес тот же


> R     центров 1-го уровня   центров 2-го уровня   кластеров волной   время,
мс
0.5     143     40     5     3.918
1     51     10     5     1.655
1.5     29     6     5     0.758
Рисунки: gnuplot plot_forel_*.gp


> Работа завершена.
```

Сборка на Linux

Все программы собираются GNU g++ 15 с флагами -Wall -Wextra -Werror -pedantic на стандартах C++11, 14, 17 и 20, с оптимизацией -O3 и без неё, без единого предупреждения. Программы добавлены в папку для флешки (08–12), их собирает build_all.sh, а run_all.sh запускает их и рисует картинки.